Как устроен курс «Методы анализа и проектирования ПО»

Ориентационная лекция по плану дисциплины Б1.О.12

Кедрин В.С., МАиППО, 2026

Викулова Елизавета

Кандидат в темы мини-проекта: пока в раздумьях

Ожидание от курса: Перейти от кода по наитию к осмысленному проектированию

Мои 3+3

Ожидания:

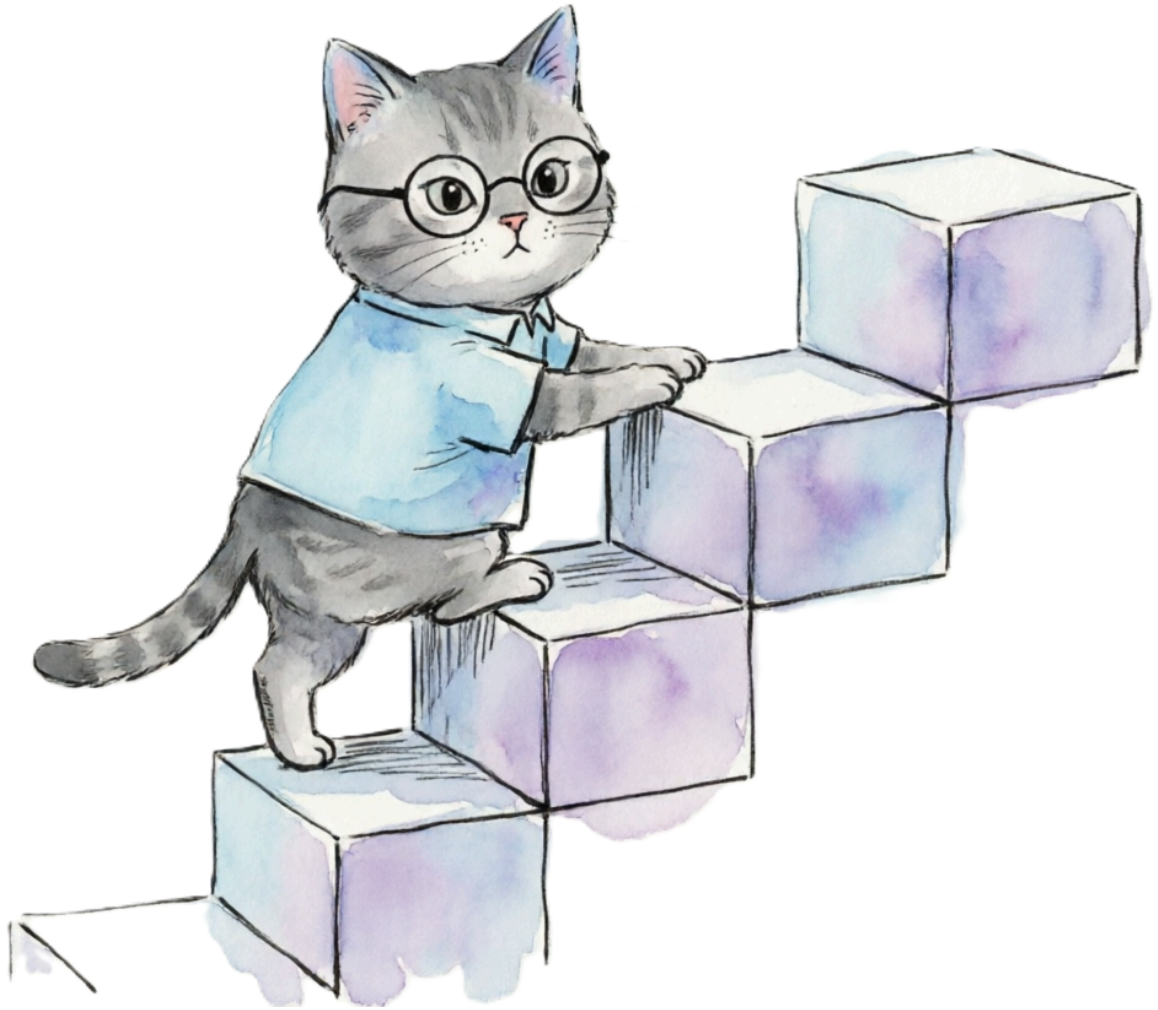
1. Развить системное мышление
2. Научиться правильно декомпозировать задачи
3. Собрать полный проект

Опасения:

1. Нехватка опыта и знаний
2. "Утонуть" теории
3. Выбрать сложную тему

План рассказа

1. Две истории о цене анализа
2. Паспорт дисциплины и инструменты
3. Цель курса и 8 умений выпускника
4. Карта 15 лекций (2 части)
5. Лабораторные и сквозной мини-проект
6. График контроля и путь к зачёту
7. Материалы курса
8. Правила игры и советы



История 1: «Маринер-1» (1962)

- Зонд к Венере, старт 22 июля 1962 г.
- Через 5 минут ракета ушла с курса → **команда самоуничтожения**
- Потери: ~10 млн долларов

Причина: в бумажной спецификации пропущена **черта над символом** в формуле

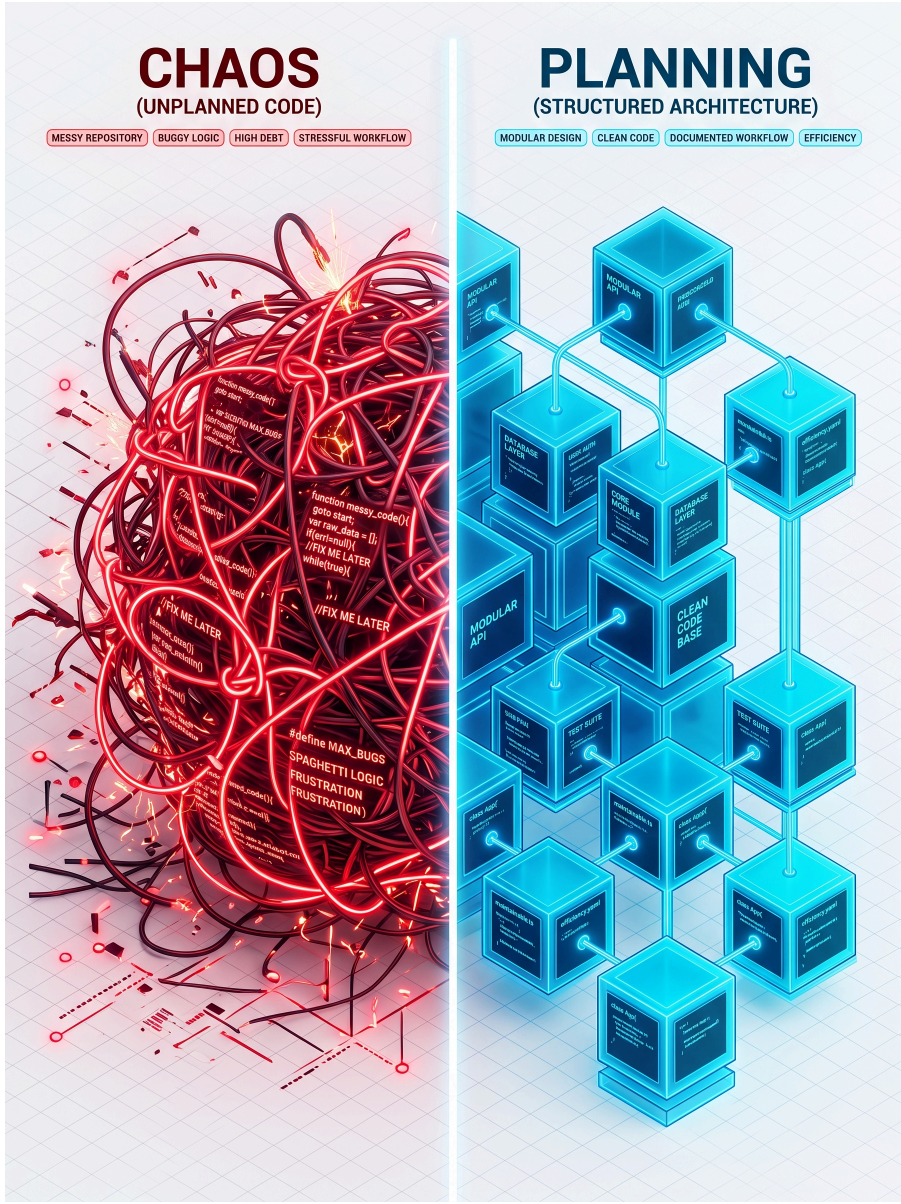
> Систему погубил не код, а дефект в **проектной документации**

История 2: «сразу писать код»

Студент прошлых потоков:

- Недели 1–5: «модели — для слабаков», 900 строк кода
- Неделя 7: код не собирается → **переписал всё с нуля**
- Неделя 15: защита — хорошо, но сил потрачено вдвое больше

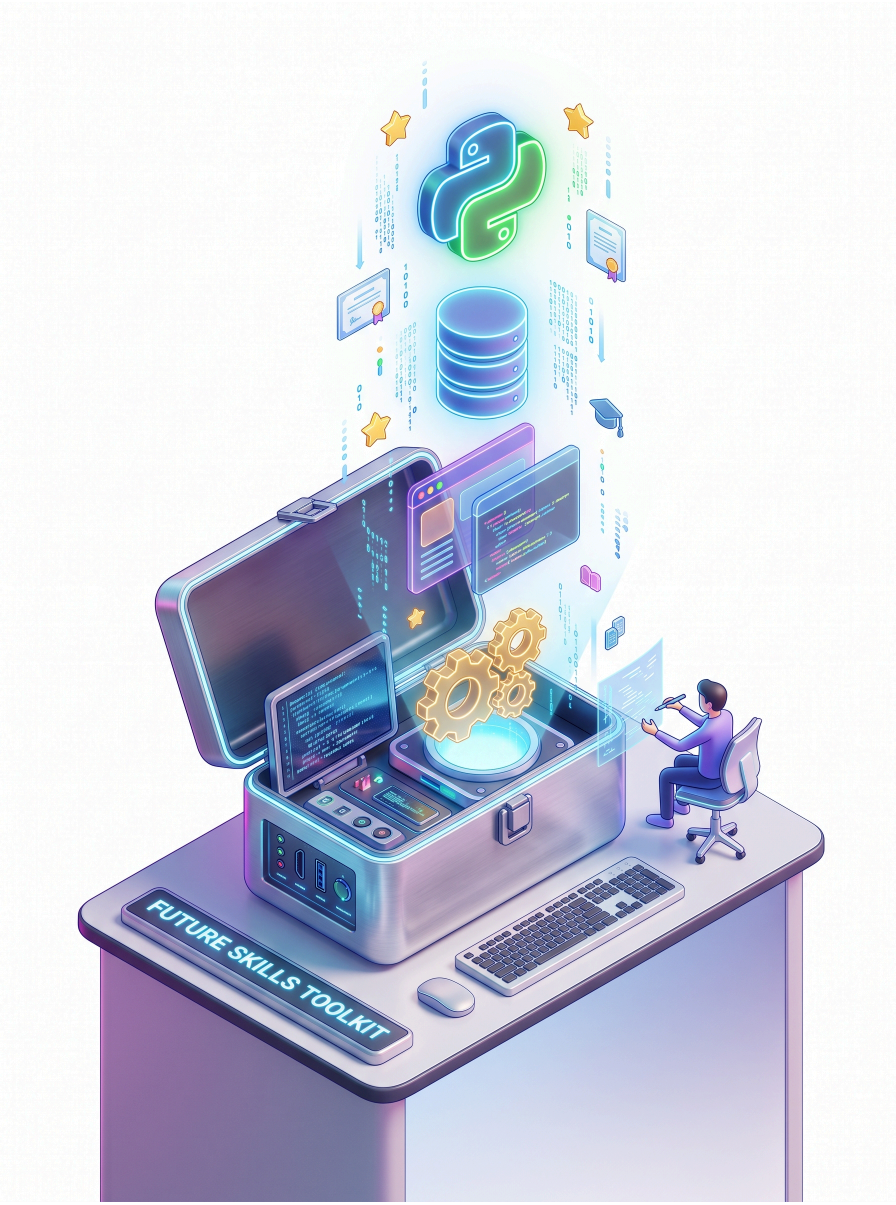
> Анализ и проектирование — способ **ошибаться на бумаге**, пока ошибки дёшевы



Паспорт дисциплины (из плана)

Параметр	Значение
Код	Б1.О.12 «Методы анализа и проектирования ПО»
Группа / семестр	02222-ДМ, 2 курс, 3 семестр (осень 2026/27)
Нагрузка	**Лекции 30 ч + Лабораторные 30 ч** (2+2 ч/нед.)
Период	15 недель: 01.09.2026 – 20.12.2026
Контроль	**зачёт**
Платформа	educa.isu.ru

Неделя теории ↔ неделя практики: **всегда в паре**



Инструменты курса

Инструмент	Роль
Python 3.x	основной язык
Jupyter	среда экспериментов
Git	история и командная работа
SQLite / PostgreSQL	базы данных
Django	веб-интерфейс проекта

Всё бесплатное; настройка — **ЛР-1**

Цель курса (из плана)

> Сформировать **систему методов** анализа, проектирования и конструирования ПО:
> от декомпозиции задачи и структурного программирования —
> через ООП, паттерны и метрики качества —
> к архитектуре приложения и **защите мини-проекта**

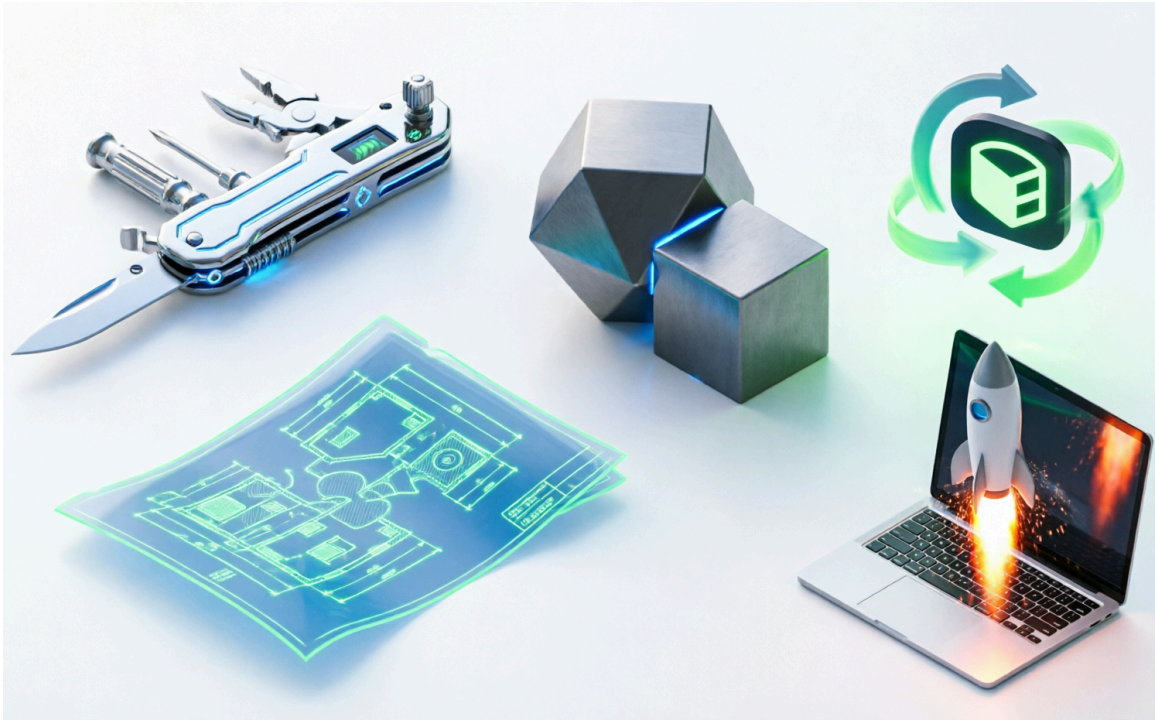
Ключевое слово — **система**: каждая ступень опирается на предыдущую

8 умений выпускника (1/2)

1. **Декомпозиция**, проектирование «сверху вниз»
   — как съесть слона: по кусочкам
2. **Структурное + контрактное** программирование
   — код с дисциплиной и договорами
3. **Тесты (doctest, unittest), TDD**
   — доказать, а не уверенно заявить
4. **Метрики сложности + рефакторинг**
   — измерить запутанность и распутать

8 умений выпускника (2/2)

5. **SOLID + UML** — зрелая архитектура и её язык
6. **Паттерны GoF** — 6 ключевых из каталога
7. **Проектирование БД** — ER → нормализация → SQL
8. **Мини-проект на Django** — сборка всего и защита



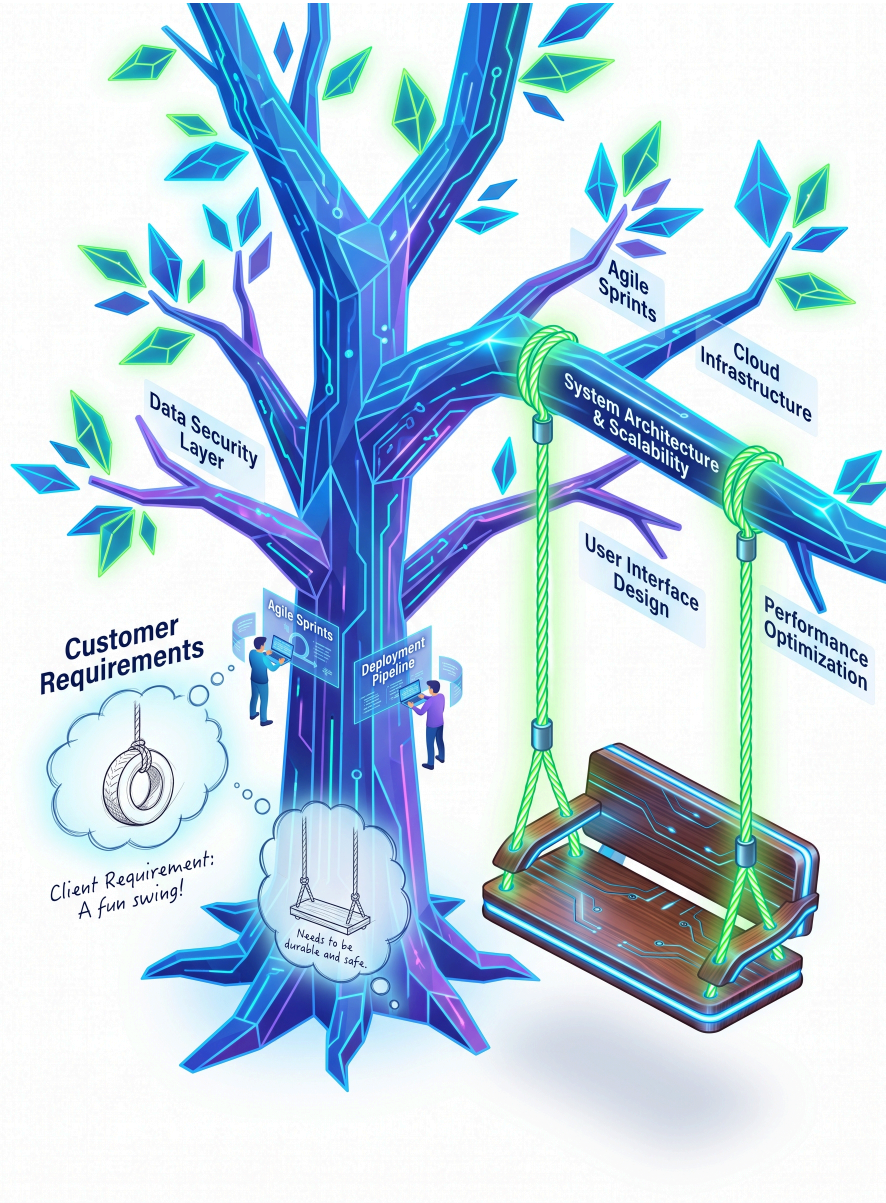
> Билл Гейтс: измерять код строками — как самолёт его весом.
> Учимся измерять правильно.

Карта лекций: недели 1–3

Нед.	Тема	Зачем
1	Введение, ЖЦ ПО, роли	карта индустрии, RACI
2	IDEF0, DFD, IDEF1X	модель = скелет будущего кода
3	Требования (ГОСТ 34, ISO 29148)	проверяемые требования, трассировка

«Качели на дереве»: заказчик → отдел → программист → пень.

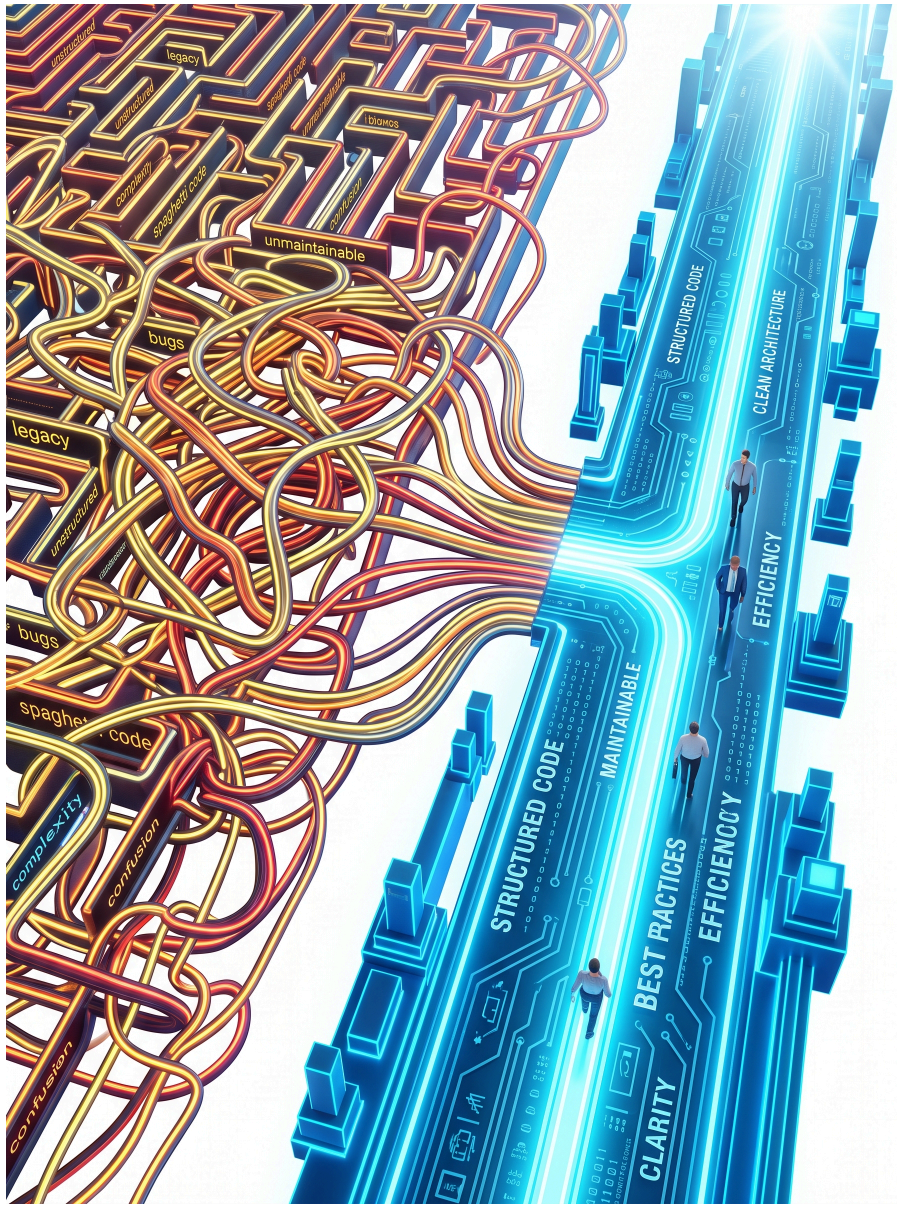
Первые 3 недели — чтобы качели остались качелями.



Карта лекций: недели 4–6

Нед.	Тема	Зачем
4	Структурное программирование, PEP 8	дисциплина кода, отказ от goto
5	Проектирование «сверху вниз»	декомпозиция в код
6	Контрактное программирование	пред/постусловия, assert, PyContracts

1968, Дейкстра: «Go To Statement Considered Harmful» —
правда победила, goto в Python нет 🙂



Карта лекций: недели 7–8

Нед.	Тема	Зачем
7	Тестирование: пирамида, виды, doctest	тест-дизайн, первые автотесты
8	unittest, моки, покрытие, **TDD**	тест раньше кода

Анекдот: заходит тестировщик в бар —
1 пиво, 0 пива, 999999 пива, −1 пиво, ящерица, qwergyu…
А пользователь спросил, где туалет — **и бар загорелся**

Итог первой половины

```
Анализ → Модель → Требования → Код → Контракты → Тесты
  1          2          3         4–5       6           7–8
```

К середине семестра у вас:

- понятая и смоделированная задача
- работающее консольное ядро
- ядро с контрактами **и под тестами**

Карта лекций: недели 9–11

Нед.	Тема	Зачем
9	Метрики (LOC, Мак-Кейб, Холстед), техдолг	споры на ревью — цифрами
10	ООП: классы, инкапсуляция, полиморфизм	от слов к инструментам
11	SOLID, abc, UML, рефакторинг	зрелый дизайн + диаграммы

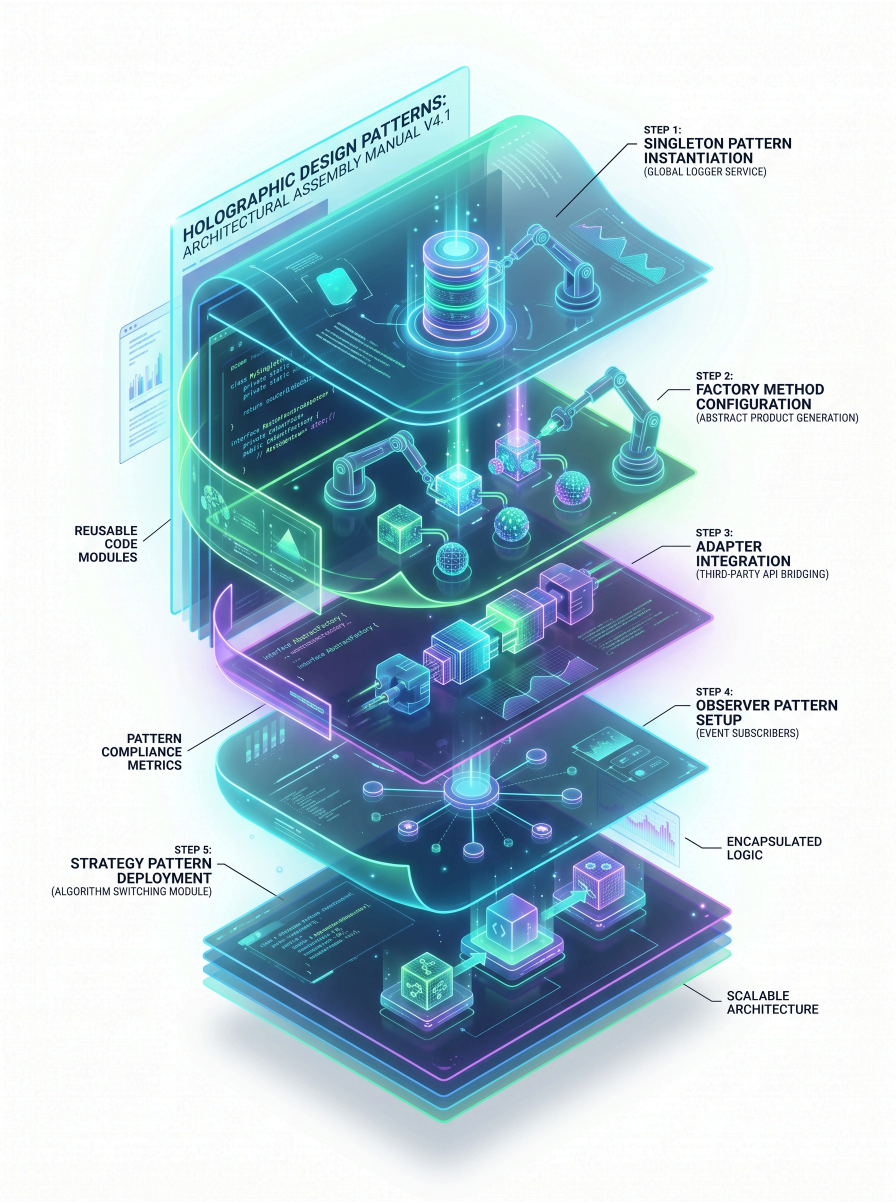
Барбара Лисков — первая женщина с премией Тьюринга;

принцип подстановки L работает в вашем проекте с 8-й недели

Карта лекций: недели 12–13

Нед.	Тема	Зачем
12	GoF: Decorator, Adapter, Observer	типовые решения структур и поведения
13	CoR, Abstract Factory + YAML	конвейеры и конфигурация

1994, «банда четырёх» (GoF): каталог из 23 паттернов —
как каталог IKEA: проблема типовая, решение проверенное



Карта лекций: недели 14–15

Нед.	Тема	Зачем
14	БД: ER, нормализация, SQL/NoSQL, ACID	данные как спроектированная система
15	MVC/MTV, веб-контур, ORM, Git, **AI в разработке**	сборка картины + итоги

Knight Capital, 2012: старый код в проде — **−440 млн $ за 45 минут**. Тесты и Git — страховка

Лабораторные: не 15 работ, а ОДИН проект

Сквозной мини-проект (из плана):

> консольное ядро → ООП/паттерны → БД → Django-интерфейс

- ЛР-1…ЛР-15 развивают **одну систему**
- Проект проживает **весь жизненный цикл**
- Финал — **защита** (первый питч вашей системы)

Эволюция проекта по неделям

Недели	Стадия
1–2	репозиторий, тема, **IDEF0-модель**
3–6	консольное ядро: структура → контракты → тесты → рефакторинг
7–10	ООП, UML, SOLID-ревью, паттерны
11–12	БД: ER → нормализация → SQL
13–15	Django-интерфейс → **защита**

Каждая стадия **опирается на предыдущую**

Примеры тем (из плана)

- 💰 личный финансовый трекер
- ✅ трекер привычек
- 📚 каталог книг / фильмов
- 🗓️ запись клиентов на услуги

Можно предложить **свою** — согласовываем

> Правило: тема должна быть **понятной**, а не модной.
> «Понять — важнее, чем впечатлить»

История: «скучная тема» побеждает

- Студент А: «скучный» каталог настольных игр →
  рекомендации «во что поиграть», статистика партий, API на телефон 🏆
- Студент Б: «нейросеть для…» →
  не декомпозируется, утонул к 7-й неделе

Выбирайте тему, с которой проживёте 15 недель **с удовольствием**

График контроля (из плана)

Неделя	Контрольная точка
3	Допуск ЛР-1, ЛР-2
6	Допуск ЛР-3, ЛР-4 — **аттестация 1**
9	ЛР-5…7 + UML-диаграмма — **аттестация 2**
12	ЛР-8…10 + схема БД — **аттестация 3**
15	ЛР-11…15, **защита мини-проекта, зачёт**

Путь к зачёту

Формула: выполненные ЛР (обязательная форма) + защита мини-проекта

Бонусы:

- опорный конспект лекции (marp)
- доклад с AI-агентом (есть готовый пример)

Консультация: **вторник, 18:30**, дистанционно

Математика долгов

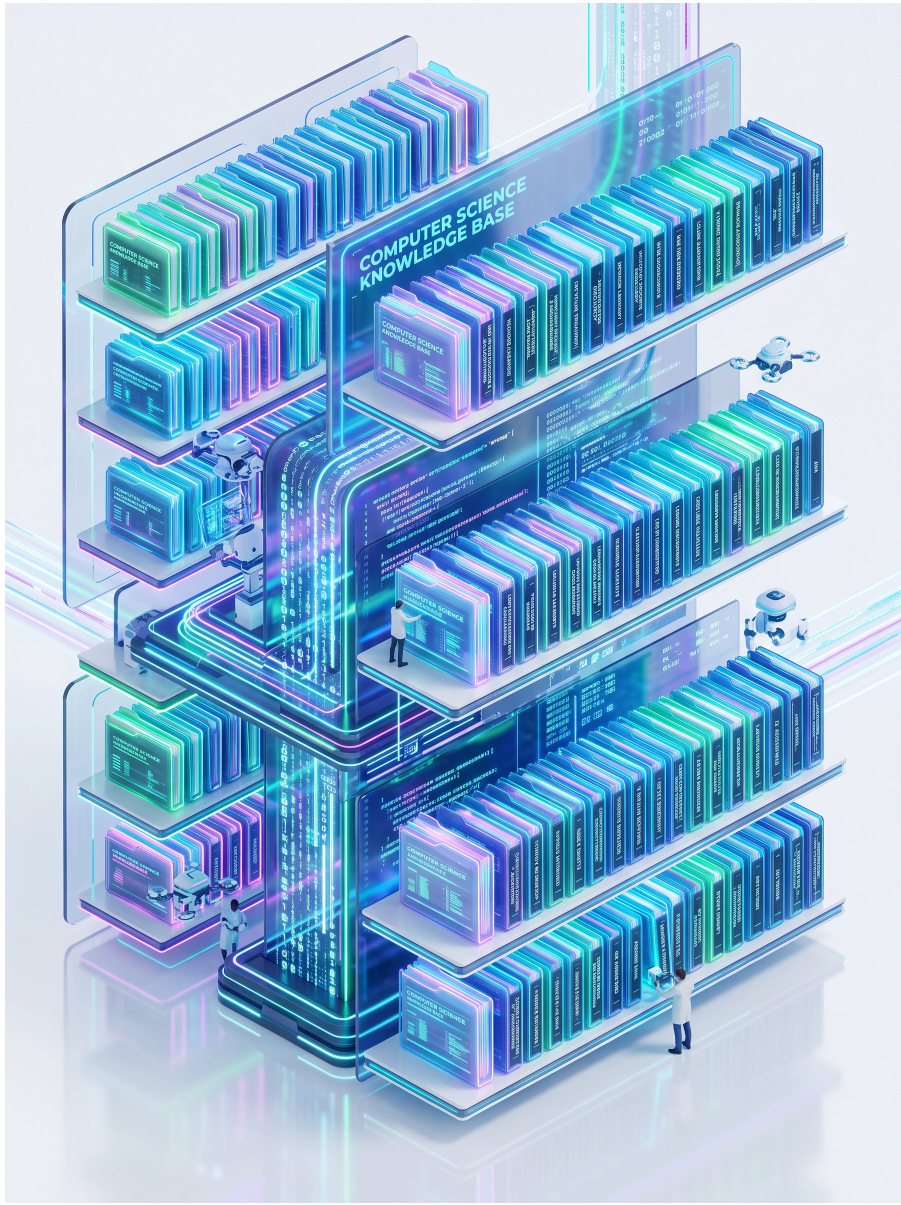
```
1 просроченная ЛР = −2 часа вашей следующей недели
2 просроченные ЛР к 10-й неделе = пятница с SQLite вместо друзей
```

Долг не рассасывается — **капитализируется**

График точек — ваша страховка от спирали

Материалы курса: каждая тема = папка

```
NN_Тема/
├── lec/  NNl_лекция_marp.md      ← слайды (Markdown → презентация)
├── lr/   NNlr_МУ_задания.md      ← цель, шаги, варианты, критерии 3/4/5
└── материалы/                     ← конспекты курсов, примеры, видео
```



Как пользоваться материалами

1. **До** лабораторной: 10 минут на МУ → «я пришёл делать», а не «чего хотят»
2. **После** лекции: расшифровка — для повторения и подготовки к зачёту
3. **Для докладов**: marp-шаблоны курса — готовая основа бонус-задания
4. **Примеры кода**: архивы в материалах — не копируйте, **разбирайте**
5. **ИИ-ассистент** — помощник, но ответственность за код — **ваша** (следующий слайд)

Как использовать ИИ в учёбе

Где помогает:

- объяснить непонятное: «объясни этот фрагмент, как для 2 курса»
- черновик кода/теста по МУ — как **отправная точка**, не финал
- тест-данные и edge-кейсы (вспомним бар с ящерицей 🦎)
- ревью-подсказка: «найди слабые места в моей функции»

Как спрашивать (качество ответа = качество запроса):

- указывайте контекст: язык, версия Python, цель, ограничения
- просите **объяснение к коду**, а не только код
- примеры «до/после» — сверяйте с МУ и конспектами

Важно:

- ✅ сгенерированное должно пройти **ваши тесты и ваше понимание**

Пять правил семестра

1. **Каждая неделя — кирпич** фундамента
2. **Репозиторий — дневник**: коммиты каждую неделю
3. **Тесты — не опция** (с 8-й недели работа без тестов не готова)
4. **Своё — ценнее модного** (тема проекта)
5. **Спрашивать — норма** (консультация вт. 18:30)

Портрет в декабре

У вас будет:

- 📁 репозиторий с историей разработки — **портфолио**
- 🌐 своё веб-приложение с БД
- 🧰 навыки: IDEF0 → требования → тесты → ООП → паттерны → БД → Django
- 🎤 опыт **защиты** проекта — первый публичный питч вашей системы

Финал: стартуем

- Открытые вопросы плана (СУБД, темы, график) — решаем за 2 недели
- **Неделя 1**: Python + Jupyter + Git, первый репозиторий
- Нагрузка честная: 2+2 ч занятий + ~2 ч самостоятельной работы в неделю

> Системы гибнут не от кода, а от пропущенной черты в проекте.
>
> **Учитесь видеть черты.**

До встречи на следующей неделе!